

BackupSage

Index, search and **deduplicate** files across large tar backup archives and directories — **without extracting anything**.

One streaming pass over each source builds a self-contained SQLite index holding full-text search, per-file metadata (size, mtime, mode), a full-content BLAKE3 hash, image dimensions, a perceptual hash and EXIF capture dates. A **master catalog** aggregates any number of these indexes so you can ask, across every backup you own at once:

- Which files exist in more than one backup — even under different names?
- Which photos are near-duplicates (edited, re-saved, resized copies)?
- Which copy is newest — by EXIF capture date, tar mtime, or archive date?
- Which backup contains the config file mentioning "postgres password"?

v1.0 is **strictly read-only over your data**: the only files it ever writes are its own `.db` indexes and the master catalog. It reports and recommends; it never touches an archive or a source directory.

- Reads `.tar`, `.tar.gz`, `.tar.zst` (detected by content, not extension) **and plain directories**
- Streams everything — no temp files, no extraction, bounded memory
- Dedup works with every archive offline (metadata lives in the catalog)
- `--json` on reporting commands: stable output for scripts (and the future web UI)

Install

```
git clone https://github.com/tom2025b/backupsage
cd backupsage
cargo build --release
# Binary at: ./target/release/backupsage
```

Quick start: find duplicates across two backups

```
backupsage index /backups/old-laptop.tar.zst /backups/photos-2024.tar.gz
backupsage master add /backups/old-laptop.tar.zst /backups/photos-2024.tar.gz
backupsage dedup
```

```
Group 1  exact · 2 copies · 2 archive(s) · 4.1 MiB reclaimable
  KEEP  photos-2024.tar.gz  DCIM/IMG_0142.JPG  4.1 MiB  2024-03-14 18:02
exif:DateTimeOriginal  4032x3024 (newest)
  dup   old-laptop.tar.zst  backup/img_0142 (1).jpg  4.1 MiB  2019-01-01 00:00
tar-mtime

Group 2  near (max dist 2) · 2 copies · 2 archive(s) · 3.8 MiB reclaimable
  KEEP  photos-2024.tar.gz  2023/07/IMG_2041.jpg  4.1 MiB  2023-07-04 19:22
exif:DateTimeOriginal  4032x3024 (highest-resolution)
  dup   old-laptop.tar.zst  dl/IMG_2041-edit.jpg  3.8 MiB  2024-01-11 08:01
tar-mtime 1600x1200 [dist 2]
```

Every line shows the best available timestamp **with its source** (EXIF capture date > tar-header mtime > archive index date) so you decide which copy survives. KEEP is advisory — nothing is ever deleted or moved.

One-off comparison without a catalog:

```
backupsage dedup --db a.tar.zst.db --db b.tar.gz.db
```

Commands

index — build a v3 index for archives or directories

```
backupsage index /backups/system.tar.zst
backupsage index ~/Pictures             # directories work too → ~/Pictures.db
```

One streaming pass per source captures: FTS5 text index (as v0.2), tar header mtime/size/mode, full-content BLAKE3 (covers every byte, even past the text cap), image width/height + perceptual hash (JPEG/PNG/WebP/TIFF), EXIF dates (JPEG, TIFF-RAW, PNG, WebP, **HEIC** — metadata needs no pixel decode), and a whole-archive fingerprint for master verify --deep. The index lands at <source>.db next to the source.

Flag	Description
--index <FILE> / -i	Custom index path (single source)
--max-file-size <SIZE>	Text-search cap per file (default 16M; hash always covers everything)
--media-cap <SIZE>	Image decode cap (default 64M; larger images get no pHash)
--no-word-stats	Skip word statistics (faster; top empty)

master — the catalog across all your backups

```
backupsage master add /backups/*.tar.zst # or the .db files
backupsage master list
backupsage master sync                  # refresh replicas of rebuilt indexes
backupsage master verify --deep         # did an archive change since indexing?
backupsage master rm old-laptop.tar.zst
```

The master (default `~/.local/share/backupsage/master.db`, override with `--master` or `BACKUPSAGE_MASTER`) replicates **metadata only** — never file content. Dedup runs entirely against it, so archives can live on unplugged drives. Staleness is tracked on two levels: replica-vs-index (`sync` fixes automatically) and index-vs-archive (`verify` reports; re-index to fix). Old v2 indexes register as `v2-limited`: searchable, but they carry no hashes — re-indexing the archive is the upgrade.

dedup — duplicate report across everything registered

```
backupsage dedup --min-size 100K --sort wasted
backupsage dedup --kind image --threshold 2 --across-only
backupsage dedup --json -o dupes.json
```

Exact groups = identical BLAKE3 content, any filename. Near groups = images within hamming distance ≤ 3 of the 64-bit DCT perceptual hash (the same semantics photo-organizer used), found via banded multi-index lookup — no $O(n^2)$ scans. Empty files, sparse tar entries and symlinks are excluded; hardlinks are shown but never counted as reclaimable; a path stored twice in one tar is reported as intra-archive waste.

Near groups are chains of pairwise matches, so a member can be farther than the threshold from the kept copy. Such transitive-only members are tagged `[review-only]` (JSON: `"actionable": false`) and excluded from reclaimable counts — only members measured directly within threshold of the keeper are ever safe for automatic action (see ADR 0004).

Flag	Description
<code>--exact-only</code> / <code>--near-only</code>	Restrict the match type
<code>--threshold 0..3</code>	Near-match strictness (default 3)
<code>--kind</code> , <code>--ext</code> , <code>--min-size</code> , <code>--path-glob</code> , <code>--archive</code>	Scope filters
<code>--across-only</code>	Hide groups confined to one archive
<code>--include-empty</code>	Opt empty files back in
<code>--db <FILE></code> (repeatable)	Ad-hoc mode without the master
<code>--sort wasted\ count\ newest</code> , <code>--limit-groups N</code>	Presentation
<code>--json</code> , <code>-o FILE</code>	Machine-readable report

Exit codes: 0 clean · 1 error · 2 completed but some archives were skipped/offline/incomplete (details in the report). The JSON shapes and exit codes are frozen by golden

fixtures — see docs/CONTRACT.md. The [compatibility policy](#) defines their v1.x support window and the process for any future breaking change.

search — full-text search, one archive or all

```
backupsage search "password" -a /backups/system.tar.zst # as v0.2
backupsage search "invoice 2023" --all                 # every registered
archive
```

--all fans out to each per-archive index (v2 included) and groups results per archive — BM25 scores are not comparable across separate indexes, so no fake global ranking. Offline archives are listed, never silently skipped. FTS5 syntax ("exact phrase", prefix*, AND/OR/NOT) as before; invalid syntax falls back to a literal search.

top — most frequent words in one index (unchanged from v0.2)

inspect — everything the index knows about one file

```
backupsage inspect DCIM/IMG_0142.JPG -a /backups/photos.tar.zst
```

Prints type, size, mtime, mode, BLAKE3, pixel dimensions, pHash, EXIF date with its source field, and flags (truncated / sparse / shadowed / ...).

Honest numbers & limits

- **Index size:** the FTS index stores a full copy of all indexed text (that's what makes snippets work) — expect roughly the size of the text content. Media contribute only ~150 bytes of metadata per file. The security note stands: a --mode full (default) .db contains plaintext from your backup — protect it like the backup itself.
- **Content modes and what they leak** (--mode full|search-only|metadata-only, #39/#70/#71): search-only drops the recoverable-plaintext copy (SQLite FTS5 contentless storage) but still stores every distinct token and its frequency — that is what makes it searchable at all. **This is not encryption and not a privacy boundary:** word lists and frequencies can leak real information about the source text. metadata-only stores no content-derived data whatsoever — no hashes, no tokens, no snippets, no dedup — only filesystem-level facts (path, size, mtime, kind). Treat search-only indexes with the same care as full ones; only metadata-only indexes are safe to handle more casually than the backup they describe.
- **Speed:** indexing is typically decompression-bound; BLAKE3 hashing adds multi-GB/s SIMD work, not I/O. Compressed archives already had to decompress every byte anyway.
- **Master size:** ~150 B per file record — 1M files ≈ 150 MB.
- **HEIC/RAW:** exact dedup, EXIF dates and timestamps work; **no perceptual hash in v1.0** (HEIC pixel decode needs libheif — planned as an opt-in feature). Reports count the affected images so the blind spot is visible.
- **Videos:** exact hash only, like photo-organizer's behavior.

- **Sparse tar entries** are flagged and excluded from dedup (upstream tar reader limitation); a corrupt tar entry still aborts loudly and marks the index incomplete — and that status follows the data into every report.
- **pHash stability**: the algorithm (`sage-dct-v1`) is frozen behind golden tests and versioned in every index; near-dup refuses to compare across different algorithm versions. Distances are comparable to Python imagehash's, not bit-identical (different image stacks — a non-goal).

Upgrading from v0.2

v2 indexes keep working for `search/top` forever, and can join federated search via `master add (v2-limited)`. Dedup needs hashes, which only exist in the archive itself: **re-indexing is the migration**, one command per archive. v0.1 databases: re-index.

Architecture

```
tar: File → BLAKE3 fingerprint → progress → BufReader → zstd/gzip/none → tar
entries
dir: walkdir ───────────────────────────────────────────────────────────────────┐
                                                                                   ▼
                                per entry: chunked read to EOF — every byte → per-file BLAKE3
                                ┌ first N bytes → [null-probe → FTS5+words | image decode
                                │   → dims+pHash | EXIF date] → <source>.db (schema v3)
                                └──────────────────────────────────────────────────────────────────┘

master.db ← master add/sync: ATTACH one .db at a time, copy metadata rows
dedup      ← runs on master only (exact: hash groups · near: 4×16-bit MIH bands)
search --all ← fan-out to per-archive FTS5, grouped results
```

- `src/format.rs` — magic-byte format detection
- `src/indexer.rs` — streaming pipeline; single-pass hash+capture
- `src/source_dir.rs` — directory walking front-end
- `src/store.rs` — schema v3, inserts, finalise post-passes
- `src/phash.rs` — frozen `sage-dct-v1` DCT perceptual hash
- `src/exif_date.rs` — EXIF dates + media kinds
- `src/master.rs` — catalog, replication, staleness
- `src/dedup.rs` — exact + near grouping, keep policy
- `src/report.rs` — the JSON contract
- `src/searcher.rs` — FTS5 queries, discovery, federation
- `src/cli.rs` / `src/main.rs` — clap definitions and rendering
- `tests/` — 75 tests over real generated archives

Design docs: `docs/superpowers/specs/` (architecture) and `docs/superpowers/plans/` (implementation plan).

Roadmap

BackupSage's risk-gated release sequence is **v1.0.1 safety** → **v1.0.2 correctness** → **v1.1 intelligence and plans** → **v1.2 controlled actions** → **v1.3 media and scale** → **v2.0 local web** → **v2.1 remote sources**. See the reader-facing [roadmap](#) for each milestone's outcome, dependencies, issue checklist, and exit gate. The permanent invariant remains: BackupSage never rewrites or deletes from an archive.

License

MIT — Thomas Lane